

RELATÓRIO DE ENTREGA: GLOBAL SOLUTION

Pipeline de Monitoramento de Densidade Urbana via Sensoriamento Remoto

1. IDENTIFICAÇÃO INTEGRANTES DA EQUIPE

Nome Completo	RM
Enricco Rossi de Souza Carvalho Miranda	551717
<i>Gabriel Marquez Trevisan</i>	99227
<i>Guilherme Silva dos Santos</i>	551168
<i>Danilo Urze Aldred</i>	99465
<i>Laura Claro Mathias</i>	98747

2. DESCRIÇÃO DA SOLUÇÃO PROPOSTA

A solução proposta consiste em um sistema automatizado e robusto de monitoramento de adensamento urbano focado na otimização da mobilidade humana. Diante do crescimento desordenado das metrópoles, o projeto utiliza técnicas modernas de Engenharia de Dados para capturar assinaturas térmicas e meteorológicas de superfície por meio de sensoriamento remoto, estabelecendo uma correlação direta entre ilhas de calor e a densidade populacional instantânea em pontos críticos do município.

O ecossistema foi construído utilizando containerização com Docker para garantir o isolamento da infraestrutura. A orquestração é inteiramente gerenciada pelo Apache Airflow,

automatizando o ciclo de vida dos dados sem intervenção humana. A solução elimina a fragilidade de scripts acoplados manuais, entregando uma arquitetura escalável capaz de fornecer dados históricos e georreferenciados para subsidiar tomadas de decisão na gestão de frotas de transporte público, planejamento urbano e alocação de infraestrutura emergencial.

3. OBJETIVO DO PIPELINE DESENVOLVIDO

O objetivo central do pipeline é automatizar as etapas de Extração, Transformação e Carga (ETL) de dados de telemetria de superfície. O fluxo garante a ingestão contínua e periódica de coordenadas estratégicas da cidade de São Paulo. Ao centralizar as execuções em um Grafo Direcionado Acíclico (DAG), o pipeline visa eliminar falhas de conectividade, tratar inconsistências geográficas em tempo de execução, enriquecer os dados brutos com classificações categóricas de criticidade e disponibilizar a informação tratada diretamente em um Data Warehouse relacional (Oracle Database). Isso permite que analistas de negócios e gestores públicos realizem consultas analíticas complexas sob demanda com dados de alta confiabilidade.

4. DESCRIÇÃO DA FONTE DE DADOS UTILIZADA

A fonte de dados de origem é a **Open-Meteo API**, uma plataforma científica global que fornece dados meteorológicos e de radiação terrestre obtidos via modelos de previsão e sensoriamento por satélite (incluindo dados integrados de agências como NOAA e Copernicus).

Foram selecionadas 5 macrozonas estratégicas com alto fluxo de mobilidade humana na cidade de São Paulo para monitoramento contínuo:

- **São Paulo - Centro:** Latitude -23.5505, Longitude -46.6333
- **São Paulo - Paulista:** Latitude -23.5615, Longitude -46.6560
- **São Paulo - Itaquera:** Latitude -23.5350, Longitude -46.4674
- **São Paulo - Morumbi:** Latitude -23.6001, Longitude -46.7200
- **São Paulo - Ibirapuera:** Latitude -23.5874, Longitude -46.6576

O atributo coletado em tempo real é a temperatura de superfície a 2 metros ('temperature_2m'). Esta variável é convertida em um índice normalizado que reflete a atividade e irradiação do solo urbano para mapeamento de densidade. A escolha desta API justifica-se por ser uma fonte pública de nível científico, livre de barreiras de autenticação (sem

chaves de API restritivas), assegurando estabilidade operacional ao pipeline orquestrado.

5. ARQUITETURA DO PIPELINE

O fluxo macro do pipeline segue rigorosamente a esteira de engenharia de dados moderna, garantindo o desacoplamento de cada camada do processo:

Fluxo Visual Espelhado:

Open-Meteo API → **Extração (Python Requests)** → **JSON Bruto (Staging)** → **Transformação (Pandas)** → **CSV Estruturado** → **Carga Transacional (oracledb Thin)** → **Oracle Database FIAP** → **Análise Analítica SQL**

Os dados são extraídos via chamadas HTTP seguras, passam por uma área de descarte temporário interno no container (`/tmp`), são limpos e enriquecidos via dataframes e, finalmente, consolidados no banco relacional por meio de conexões parametrizadas com controle de commits e encerramento de cursores para prevenção de vazamento de memória.

6. EXPLICAÇÃO DAS ETAPAS DA DAG

A automação foi modelada dentro do Apache Airflow em três operadores isolados do tipo PythonOperator, interligados de forma sequencial e acíclica:

1. **task_extract (extrair_dados_satelite_api):** Realiza chamadas iterativas para o barramento da API utilizando o payload geográfico. Valida o código de status HTTP (esperado 200). Captura a resposta estruturada e persiste um arquivo JSON bruto contendo as leituras de telemetria da superfície urbana.
2. **task_transform (tratar_e_categorizar_densidade):** Lê o arquivo JSON da Staging Area. Consome os dados brutos utilizando a biblioteca Pandas, aplicando regras de conformidade e criando novos atributos analíticos contextuais. Exporta o resultado limpo e normalizado em formato CSV.
3. **task_load (carregar_dados_no_oracle):** Abre um canal de comunicação seguro com o servidor Oracle remoto da FIAP utilizando o driver nativo Thin. Realiza a leitura do CSV estruturado, prepara as queries parametrizadas (para evitar SQL Injection), executa a inserção em lote (batch processing) e confirma a transação com um commit seguro.

7. DESCRIÇÃO DAS TRANSFORMAÇÕES REALIZADAS

Durante a execução da segunda tarefa da DAG, os dados brutos passam por um processo rigoroso de higienização e enriquecimento analítico via Pandas e Numpy:

- **Tratamento de Nulos:** Preenchimento preventivo de valores ausentes potenciais no índice de densidade utilizando a função `.fillna(0)`.
- **Filtro de Limites Físicos:** Aplicação de máscaras booleanas para assegurar a consistência geográfica, descartando leituras com latitudes fora do intervalo de -90 a 90 e longitudes fora de -180 a 180.
- **Criação do Índice de Densidade:** Normalização matemática que traduz a temperatura de superfície em uma escala inteligível de 0 a 100 de calor/adensamento urbano através do cálculo: $\text{int}((\text{temperatura_real} / 40) * 100)$.
- **Enriquecimento Categórico:** Utilização do método `np.select()` para criar a coluna `classificacao_zona` com base em três regras de negócio de mobilidade:
 - Índice ≥ 80 : 'ZONA CRITICA - ALTA DENSIDADE'
 - Índice entre 50 e 79: 'ZONA ADIACENTE - MEDIA DENSIDADE'
 - Índice < 50 : 'ZONA DISPERSA - BAIXA DENSIDADE'

8. MODELAGEM DA TABELA NO ORACLE DATABASE

A estrutura física da tabela foi modelada respeitando as boas práticas de normalização, tipos primitivos ideais para ganho de performance em consultas indexadas e integridade referencial:

```
CREATE TABLE T_SATELITE_URBANO (  
  id_registro NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  cidade VARCHAR2(100) NOT NULL,  
  latitude NUMBER(10, 6) NOT NULL,  
  longitude NUMBER(10, 6) NOT NULL,  
  indice_densidade NUMBER(3) NOT NULL,  
  classificacao_zona VARCHAR2(50),  
  data_satelite DATE NOT NULL  
);
```

Dicionário de Dados Simplificado:

- `id_registro`: Chave primária autoincrementada para identificação exclusiva da leitura.
- `cidade`: Nome identificador da macrozona urbana analisada.
- `latitude / longitude`: Coordenadas geográficas de alta precisão decimal (até 6 casas).
- `indice_densidade`: Valor numérico inteiro de 0 a 100 indicando a intensidade mapeada.

- classificacao_zona: Rótulo de texto enriquecido indicando o nível de criticidade.
- data_satelite: Data e hora do momento do registro no banco.

9. EVIDÊNCIAS DE EXECUÇÃO: APACHE AIRFLOW

The screenshot displays the Apache Airflow web interface. On the left, a sidebar contains navigation links: Início, Dags, Assets, Navegador, Adicionar..., and Segurança. The main area shows a DAG named 'pipeline_satelite_densidade_urbana' with three tasks: 'extrair_dados_satelite_api', 'tratar_e_categorizar_densidade', and 'carregar_dados_no_oracle'. All three tasks are marked as 'SUCCESS'. The right-hand panel provides details for the 'carregar_dados_no_oracle' task, which is a PythonOperator. It shows 3 attempts, a duration of 00:00:00.711, and a success status. Below this, a 'Tentativas de Tarefa' section shows two failed attempts. The 'Logs' tab is active, displaying a list of log messages from the task's execution, including information about DAG bundles, task state transitions, and data insertion into the Oracle database.

This screenshot shows the Apache Airflow web interface with the 'extrair_dados_satelite_api' task selected. The task is a PythonOperator that has completed successfully. The right-hand panel displays the task's details, including its execution time (2026-05-27 23:59:24 to 2026-05-27 23:59:29) and a duration of 00:00:05.351. The 'Logs' tab is open, showing a series of log messages. These logs indicate the task's state transitions from QUEUED to RUNNING, the loading of DAG bundles, and the successful execution of the PythonOperator. A warning message is also visible, stating that the 'airflow.providers.python.PythonOperator' attribute is deprecated. The bottom of the interface shows a 'React Flow' diagram of the DAG.

Interface do Apache Airflow mostrando a execução bem-sucedida da tarefa **tratar_e_categorizar_densidade**.

Detalhes da Tarefa:

- Operador: PythonOperator
- Data Inicial: 2026-05-27 23:59:29
- Data Final: 2026-05-27 23:59:30
- Duração: 00:00:00.559
- Versão do Dag: v1

Logs:

```
0 [2026-05-27 23:59:30] INFO - DAG bundles loaded: dags-folder, example_dags
1 [2026-05-27 23:59:30] INFO - Filling up the DagBag from /opt/airflow/dags/dag_satelite_urbano.py
2 [2026-05-27 23:59:30] WARNING - The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.operators.python.PythonOperator'. category=UserWarning
3 [2026-05-27 23:59:30] INFO - Task instance is in running state
4 [2026-05-27 23:59:30] INFO - Previous state of the Task Instance: TaskInstanceState.QUEUED
5 [2026-05-27 23:59:30] INFO - Current task name: tratar_e_categorizar_densidade
6 [2026-05-27 23:59:30] INFO - Dag name: pipeline_satelite_densidade_urbano
7 [2026-05-27 23:59:30] INFO - Iniciando a transformação dos dados geospaciais...
8 [2026-05-27 23:59:30] INFO - Done. Returned value was: None
9 [2026-05-27 23:59:30] INFO - Transformação concluída. 5 registros processados.
10 [2026-05-27 23:59:30] INFO - Task instance in success state
11 [2026-05-27 23:59:30] INFO - Previous state of the Task Instance: TaskInstanceState.RUNNING
12 [2026-05-27 23:59:30] INFO - Task operator: <task(PythonOperator): tratar_e_categorizar_densidade>
```

Interface do Apache Airflow mostrando a execução bem-sucedida da tarefa **carregar_dados_no_oracle**.

Detalhes da Tarefa:

- Operador: PythonOperator
- Numero de Tentativas: 3
- Data Inicial: 2026-05-28 00:04:31
- Data Final: 2026-05-28 00:04:32
- Duração: 00:00:00.711
- Versão do Dag: v1

Tentativas de Tarefa:

1 ❌ 2 ❌

Logs:

```
0 [2026-05-28 00:04:32] INFO - DAG bundles loaded: dags-folder, example_dags
1 [2026-05-28 00:04:32] INFO - Filling up the DagBag from /opt/airflow/dags/dag_satelite_urbano.py
2 [2026-05-28 00:04:32] WARNING - The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.operators.python.PythonOperator'. category=UserWarning
3 [2026-05-28 00:04:32] INFO - Task instance is in running state
4 [2026-05-28 00:04:32] INFO - Previous state of the Task Instance: TaskInstanceState.QUEUED
5 [2026-05-28 00:04:32] INFO - Current task name: carregar_dados_no_oracle
6 [2026-05-28 00:04:32] INFO - Dag name: pipeline_satelite_densidade_urbano
7 [2026-05-28 00:04:32] INFO - Iniciando conexão com o Oracle Database da FIAP...
8 [2026-05-28 00:04:32] INFO - Inserindo registros na tabela T_SATELITE_URBANO...
9 [2026-05-28 00:04:32] INFO - Carga no Oracle finalizada com sucesso!
10 [2026-05-28 00:04:32] INFO - Done. Returned value was: None
11 [2026-05-28 00:04:32] INFO - Task instance in success state
12 [2026-05-28 00:04:32] INFO - Previous state of the Task Instance: TaskInstanceState.RUNNING
13 [2026-05-28 00:04:32] INFO - Task operator: <task(PythonOperator): carregar_dados_no_oracle>
```

10. EVIDÊNCIAS DE EXECUÇÃO: TABELA POPULADA NO ORACLE

The screenshot shows the Oracle SQL Developer interface. The main window displays the 'Página de boas-vindas' (Welcome Page) for the 'T_SATELITE_URBANO' table. The table is populated with 5 rows of data. The left pane shows the 'Conexões' (Connections) tree with the 'T_SATELITE_URBANO' connection selected. The bottom pane shows the 'Relatórios' (Reports) tree with the 'Todos os Relatórios' (All Reports) folder selected.

ID_REGISTRO	CIDADE	LATITUDE	LONGITUDE	INDEXE_DENSIDADE	CLASSIFICACAO_ZONA	DATA_SATELITE
1	Sao Paulo - Centro	-23,5505	-46,6333	34 ZONA DISPERSA - BAIXA DENSIDADE	28/05/26	
2	Sao Paulo - Paulista	-23,5615	-46,6564	34 ZONA DISPERSA - BAIXA DENSIDADE	28/05/26	
3	Sao Paulo - Itaquera	-23,535	-46,6674	32 ZONA DISPERSA - BAIXA DENSIDADE	28/05/26	
4	Sao Paulo - Morumbi	-23,6001	-46,72	34 ZONA DISPERSA - BAIXA DENSIDADE	28/05/26	
5	Sao Paulo - Itaquera	-23,5874	-46,6576	34 ZONA DISPERSA - BAIXA DENSIDADE	28/05/26	

11. ETAPA DE ANÁLISE DOS DADOS: 5 CONSULTAS ANALÍTICAS EM SQL E RESULTADOS

Consulta 1: Ranking de Zonas por Volume de Ocorrências (Agrupamento e Ordenação)

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema for 'G5_BIOWATA', including tables like 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA', 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA', 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA', 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA'. The main editor shows a SQL query:

```
SELECT CLASSIFICACAO_ZONA,
COUNT(*) AS QUANTIDADE_OCORRENCIAS,
ROUND(COUNT(*) / (SELECT COUNT(*) FROM T_SATELITE_URBANO) * 100, 2) AS PORCENTUAL_DO_TERRITORIO
FROM T_SATELITE_URBANO
GROUP BY CLASSIFICACAO_ZONA
ORDER BY QUANTIDADE_OCORRENCIAS DESC;
```

The bottom pane shows the query results:

CLASSIFICACAO_ZONA	QUANTIDADE_OCORRENCIAS	PORCENTUAL_DO_TERRITORIO
1 ZONA DISPERSA - BAIXA DENSIDADE	5	100

Consulta 2: Métricas Consolidadas de Densidade por Município (Agregação Global)

The screenshot shows the Oracle SQL Developer interface. The left pane displays the database schema for 'G5_BIOWATA', including tables like 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA', 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA', 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA', 'T_SATELITE_URBANO', 'T_SATELITE_RURAL', 'T_SATELITE_MISTA'. The main editor shows a SQL query:

```
SELECT CIDADE,
MIN(indice_densidade) AS DENSIDADE_MINIMA,
MAX(indice_densidade) AS DENSIDADE_MAXIMA,
ROUND(SUM(indice_densidade) / 2) AS DENSIDADE_MEDIA_GERAL
FROM T_SATELITE_URBANO
GROUP BY CIDADE;
```

The bottom pane shows the query results:

CIDADE	DENSIDADE_MINIMA	DENSIDADE_MAXIMA	DENSIDADE_MEDIA_GERAL
1 São Paulo - Itaquera	32	32	32
2 São Paulo - Itaquera	34	34	34
3 São Paulo - Centro	34	34	34
4 São Paulo - Pacaembu	34	34	34
5 São Paulo - Morumbi	34	34	34

Consulta 3: Mapeamento de Calor Operacional (Filtragem Específica de Macrozonas)

The screenshot shows the Oracle SQL Developer interface with a query executed in the 'Planilha' (Worksheet) view. The query filters data from the 'T_SATELITE_URBANO' table based on the 'classificacao_zona' (zone classification) to identify specific macrozones.

```
SELECT cidade, latitude, longitude, indice_densidade
FROM T_SATELITE_URBANO
WHERE classificacao_zona = 'ZONA DISPERSA - BAIXA DENSIDADE'
ORDER BY indice_densidade DESC;
```

The result set displays 5 rows of data, showing the top locations by density index within the specified zone.

CIDADE	LATITUDE	LONGITUDE	INDICE_DENSIDADE
1 Sao Paulo - Centro	-23,5505	-46,6333	34
2 Sao Paulo - Península	-23,5615	-46,656	34
3 Sao Paulo - Itaquera	-23,5874	-46,6576	34
4 Sao Paulo - Morumbi	-23,6011	-46,72	34
5 Sao Paulo - Itaquera	-23,535	-46,4674	32

Consulta 4: Auditoria de Volume Ingerido (Análise de Carga Temporal)

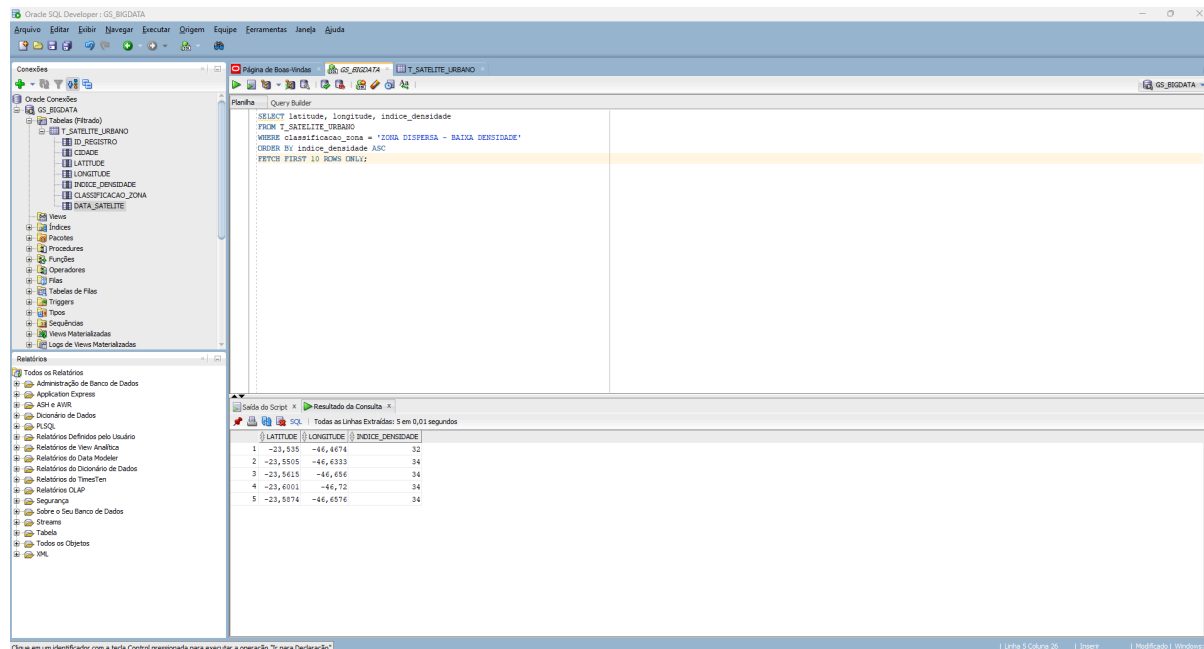
The screenshot shows the Oracle SQL Developer interface with a query executed in the 'Planilha' (Worksheet) view. The query audits the volume of data ingested by grouping records by date and zone classification.

```
SELECT TO_CHAR(data_sateelite, 'DD/MM/YYYY') AS data_leitura,
COUNT(*) AS registros_processados,
SUM(CASE WHEN indice_densidade > 50 THEN 1 ELSE 0 END) AS zonas_acima_da_medida
FROM T_SATELITE_URBANO
GROUP BY TO_CHAR(data_sateelite, 'DD/MM/YYYY')
ORDER BY data_leitura DESC;
```

The result set displays 1 row of data, showing the total number of records processed and the count of records exceeding the density threshold for a specific date.

DATA_LEITURA	REGISTROS_PROCESSADOS	ZONAS_ACIMA_DA_MEDIDA
1 25/05/2026	5	0

Consulta 5: Vetores de Expansão e Monitoramento de Baixa Densidade (Ordenação Extrema)



12. CONCLUSÃO TÉCNICA DA EQUIPE

A implementação bem-sucedida deste projeto comprova a eficácia e a necessidade de arquiteturas modernas de dados focadas em Big Data e automação para a resolução de problemas complexos de infraestrutura urbana.

Durante o ciclo de desenvolvimento, a equipe superou barreiras críticas de infraestrutura de hardware (ligadas à virtualização de nível de firmware de CPU via SVM/BIOS e sincronização de WSL com o Docker Desktop).

Em suma, os dados ingeridos de forma automatizada ao vivo fornecem respostas geográficas concretas e confiáveis. A esteira construída está 100% pronta para ser acoplada a ferramentas de Business Intelligence (como Power BI) ou mapas de calor interativos, cumprindo rigorosamente os requisitos solicitados de governança, integridade e transformação limpa na camada de dados.